

MediTrack Assignment — Prompt Handbook

Ready prompts to refactor and extend the MediTrack clinic API

Use these prompts as you work through the MediTrack assignment. They follow the same pattern as the course: each states the intent and the MediTrack-specific detail, and leaves the conventions to your CLAUDE.md and rules files. Adapt them freely — they are starting points, not a script.

Assignment 1 — Generate CLAUDE.md & Rules

IN CLAUDE CODE — Generate CLAUDE.md with /init · Module 2, Lecture 2.2: Why CLAUDE.md First

/init

PROMPT — Tighten the generated CLAUDE.md · Module 2, Lecture 2.2: Why CLAUDE.md First

Update CLAUDE.md so it requires, for this project:

- constructor injection with final fields (never @Autowired)
- BigDecimal for money (never double)
- DTOs in and out of controllers (never expose JPA entities)
- thin controllers; all business logic in a service layer
- all endpoints under /api/v1
- centralised exception handling returning a consistent ErrorResponse JSON
- ddl-auto=validate (tables come from meditrack_schema.sql)

Tip: You can reuse the course's testing.md and api-design.md rule files unchanged — the conventions are identical. Drop them into .claude/rules/.

PROMPT — Verify CLAUDE.md Is Working · Module 2, Lecture 2.2: Why CLAUDE.md First

1. What do we use for dependency injection?
2. What type do we use for money fields?

Assignment 2 — Tests & Docs

PROMPT — Unit Tests · Module 3, Lecture 3.5: Unit Tests

Generate unit tests for the service logic: mock the repositories and cover doctor not found, patient not found, and the booking and cancel paths. Add feedback cases later, as you build that feature.

PROMPT — Swagger UI · Module 3, Lecture 3.6: Swagger UI

Add interactive Swagger UI using the latest springdoc-openapi. Group the endpoints under Specialty, Doctor, Patient, and Appointment Management.

Assignment 3 — Refactor the Starter

PROMPT — Money to BigDecimal · Module 4, Lecture 4.3: Refinement Prompts

Replace every double money field – consultation fee, appointment total, service unit price and subtotal, payment amount – with BigDecimal across the models, and fix all arithmetic and JSON output. Show diffs.

PROMPT — Introduce DTOs · Module 4, Lecture 4.3: Refinement Prompts

Add request and response DTOs for specialties, doctors, patients, and appointments. Controllers must accept and return DTOs, never JPA entities; do the mapping in the service layer.

PROMPT — Extract a Service Layer · Module 4, Lecture 4.3: Refinement Prompts

Move all business logic out of the controllers into service classes. A controller should only validate input, call a service, and return a DTO. Start with the appointment booking and cancel logic.

PROMPT — Add Exceptions and a Global Handler · Module 4, Lecture 4.3: Refinement Prompts

Add ResourceNotFoundException (404), BusinessException (422), and DuplicateResourceException (409), plus a GlobalExceptionHandler that returns a consistent ErrorResponse JSON. Replace the generic RuntimeExceptions and the null returns on missing rows.

TERMINAL — Commit after each change

```
git add -A
git commit -m 'refactor: <what you changed>'
```

Assignment 4 — Patient Feedback

Note: Patient Feedback needs a new table. Add a patient_feedback table (id, appointment_id, patient_id, rating, comment, created_at; unique per appointment) to meditrack_schema.sql and run it before building, since ddl-auto=validate needs it to exist.

PROMPT — Plan Patient Feedback (plan mode) · Module 4, Lecture 4.4: Feature Workflow — Plan & Build

Enter plan mode.
Add Patient Feedback: a rating 1-5 and an optional comment on a COMPLETED appointment, one per patient per appointment, plus an average rating per doctor.

TERMINAL — Commit

```
git add -A
git commit -m 'feat(feedback): add patient feedback end-to-end'
```

Assignment 5 — Custom Commands / Skills

PROMPT — Custom Commands / Skills · Module 6, Lecture 6.1: Custom Slash Commands

```
Create slash commands in .claude/commands/:  
check-pii - flag any endpoint or log exposing patient email/phone
```

Assignment 6 — Hooks

PROMPT — Hooks · Module 7, Lecture 7.1: Register Hooks

Create a hook script: a PreToolUse guardrail that blocks writing patient email or phone into a log statement.
Register it in .claude/settings.local.json.

Assignment 7 — Playwright Smoke Test

PROMPT — Playwright Smoke Test · Module 9, Lecture 9.2: Playwright Connector

Use the Playwright connector to open `http://localhost:8080/api/v1/doctors` and confirm the response is valid JSON matching the `DoctorResponse` fields.

Assignment 8 — Git, Docker & CI

PROMPT — Git, Docker, CI · Module 10: Git, Docker & CI/CD

Commit in conventional format and push to GitHub. Then add a multi-stage Dockerfile, a docker-compose with MySQL plus `meditrack_schema.sql` as an init script, and a GitHub Actions workflow that runs the tests against a MySQL service container.